大语言模型：训练、精调与对齐 | 第 7 章（上）

强化学习

第 7 章（上）

第 9 课时 本课时讲解 7.1–7.5

强化学习基础 · 动态规划 · 时序差分 · 策略梯度

著者
邓杭　杨耀东

教材
《大语言模型：训练、精调与对齐》

出版
清华大学出版社



本章目录 第 9 课时

7.1 强化学习 第 3 页
MDP、回报、价值函数与贝尔曼方程

7.2 强化学习方法 第 31 页
三大类方法的全景与表 7.1

7.3 动态规划 第 36 页
策略评估、策略改进与 GPI

7.4 蒙特卡洛方法 第 47 页
用完整幕的平均回报估计价值

7.5 时序差分学习 第 59 页
TD、SARSA、Q 学习、DQN 与 TD(λ)

7.6 策略梯度方法 第 88 页
直接优化参数化策略：定理、证明与 REINFORCE

7.7 演员-评论家 第 109 页
演员更新策略、评论家评估价值：AC、TRPO、GAE 与 PPO

7.8 大语言模型的 MDP 描述 第 125 页
状态是词元序列，动作是下一个词元

7.9 小结 第 129 页
全章脉络：从 MDP 到三类求解方法

本课时内容 7.1 强化学习、7.2 强化学习方法、7.3 动态规划、7.4 蒙特卡洛方法、7.5 时序差分学习（其余部分见第 10 课时）

7.1

强化学习

智能体在环境中试错，学习使长期累积奖励最大的策略；MDP 给出问题的数学形式，贝尔曼方程给出求解价值函数的递归工具。

本节内容

7.1.1 强化学习介绍

7.1.2 马尔可夫决策过程

7.1.3 强化学习关键概念

7.1.4 策略和价值函数

7.1.5 贝尔曼方程

语言模型的生成过程

大语言模型逐个词元地生成文本，每生成一个词元都改变后续可选的词元集合，这正是序贯决策。因此可以把生成过程视为马尔可夫决策过程，用强化学习优化语言模型。

有监督学习

依赖带标签的静态数据集，拟合已有数据模式。

优化目标是最小化与标签之间的损失，训练信号逐词元给定。

对应第 5 章的有监督微调。

强化学习

依赖一个目标（奖励函数），对模型自己生成的样本做优化。

奖励可以只在序列结束时给出，衡量整段输出的好坏。

对应第 8 章的 RLHF 与推理强化学习。

本章建立强化学习的知识体系：先把问题写成 MDP，再依次讲清基于价值、基于策略与演员-评论家三类求解方法。

智能体与环境的交互

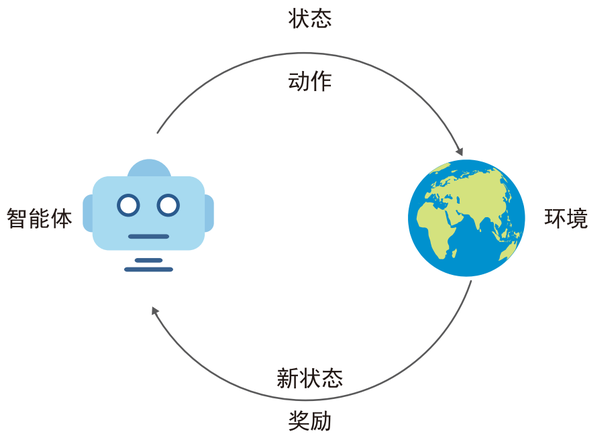


图 7.1　智能体与环境进行交互

- 每一步，智能体感知环境的状态 S_t，按策略选择动作 A_t。
- 环境响应动作，转移到新状态 S_{t+1}，并反馈奖励 R_{t+1}。
- 智能体依据奖励改进策略，指导后续的动作选择。

智能体可以是物理实体（机器人、自动驾驶汽车），也可以是软件实体（游戏 AI、交易算法）。

这条回路每转一圈产生一条四元组 $(S_t, A_t, R_{t+1}, S_{t+1})$，后续所有算法都从这样的数据里学习。

强化学习的问题定义

问题来源

复杂序贯决策

每个决策不仅影响即时结果，更会改变未来状态和长期回报，需要系统化的求解框架。

建模工具

马尔可夫决策过程

用状态、动作、状态转移概率、奖励函数和折扣因子形式化描述环境，是强化学习的理论基础。

求解对象

策略或价值函数

算法通过与环境的交互，学习策略或价值函数，而不是拟合静态数据集。

难点 环境动态具有随机性，要求最大化回报的期望；而当前决策会连锁影响未来，精确评估所有可能的决策序列会遭遇组合爆炸（维数灾难）。算法只能依靠有限次交互，学到一个能获得较高长期累积奖励的策略或价值函数。

本章安排：MDP 与关键要素（7.1.2–7.1.4）→ 贝尔曼方程（7.1.5）→ 方法分类（7.2）→ 动态规划、蒙特卡洛、时序差分（7.3–7.5）→ 策略梯度（7.6）→ 演员-评论家（7.7）→ 语言模型的 MDP 描述（7.8）。

马尔可夫链

马尔可夫链描述一个随机过程：系统的未来状态只依赖当前状态，与过去的历史状态无关。这一性质称为马尔可夫性质。

$$p(S_{t+1} \mid S_t) = p(S_{t+1} \mid S_1, S_2, \cdots, S_t) \tag{7-1}$$

状态集合 $\mathcal{S}$ 包含所有可能的状态，$p(s' \mid s)$ 为状态转移概率；对任意历史序列 $(S_1, S_2, \cdots, S_t)$，转移到 S_{t+1} 的概率只与 S_t 有关。

例 1

智能体的当前位置已经隐含了到达此处的历史路径，只需当前状态即可预测下一步转移。

例 2

预测明天的天气只需今天的天气状态，与前几天的天气历史无关。

依据 《大语言模型：训练、精调与对齐》第 7.1.2 节（式 (7-1)）

有限状态空间的描述

$$\mathcal{P} = \begin{bmatrix} p_{11} & \cdots & p_{1n} \\ \vdots & \ddots & \vdots \\ p_{n1} & \cdots & p_{nn} \end{bmatrix} \tag{7-2}$$

共有 n 个状态，$\mathcal{P} \in \mathbb{R}^{n \times n}$；$p_{ij} = p(S_{t+1} = j \mid S_t = i)$ 表示从状态 i 转移到状态 j 的概率，矩阵每行之和为 1。

- **不可约**：从任意状态出发，经过有限步可以到达任意其他状态。
- **非周期**：状态没有固定的返回周期。
- 同时满足这两个条件时，状态转移分布收敛于唯一的稳态分布。

稳态分布是 7.6 节策略梯度定理的关键：目标函数的梯度只与状态被访问的频率有关，而与状态转移概率的具体形式无关。

矩阵性质

每行是一个状态出发的转移分布，行和为 1；矩阵不要求对称。

收敛条件

不可约 + 非周期，两个条件缺一不可，否则可能出现周期振荡或不唯一。

与 MDP 的关系

MDP 的转移还要以动作为条件，即 $p(s' \mid s, a)$，相当于每个动作一张矩阵。

马尔可夫决策过程

马尔可夫决策过程（MDP）是描述序列决策问题的数学模型，其目标是在随机环境中寻找最优策略，最大化从当前状态开始的期望累积折扣奖励。一个有限 MDP 由五元组 $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ 定义。

$\mathcal{S}$

有限的状态集合，t 时刻的状态记为 S_t。

$\mathcal{A}$

有限的动作集合，t 时刻的动作记为 A_t。

P

$p(s' \mid s, a) = \Pr(S_{t+1} = s' \mid S_t = s, A_t = a)$。

R

奖励函数，在状态 s 执行 a 转移到 s' 得到 $r(s, a, s')$。

γ

折扣因子 $\gamma \in [0, 1]$，把未来奖励折算为当前价值。

核心

状态转移概率刻画环境对动作的随机响应，是建模不确定性的核心。

确定性转移是随机性转移的特例，此时 $p(s' \mid s, a) \in \{0, 1\}$。例如网格环境中，若 $S_{(1,1)}$ 的左方可通行，则 $P(S_{(1,0)} \mid S_{(1,1)}, A_{左}) = 1$。

依据 《大语言模型：训练、精调与对齐》第 7.1.2 节（MDP 五元组）

MDP 与强化学习的关系

MDP 提供形式化框架

为强化学习问题提供严谨的数学描述，定义了状态、动作、转移、奖励和折扣等核心要素。

它回答“环境是什么样的”。

强化学习提供求解方法

通过智能体与环境的交互，学习最优策略以最大化累积奖励，其本质就是求解一个 MDP。

它回答“智能体怎样学到好行为”。

环境模型

状态、动作、转移、奖励、折扣

→

交互采样

策略 π 产生 $(S_t, A_t, R_{t+1}, S_{t+1})$

→

策略 / 价值函数

估计、改进、再采样

结论 MDP 是强化学习的理论基础，强化学习是解决 MDP 问题的实践方法学，两者共同构成处理序列决策问题的完整体系。

序列决策任务

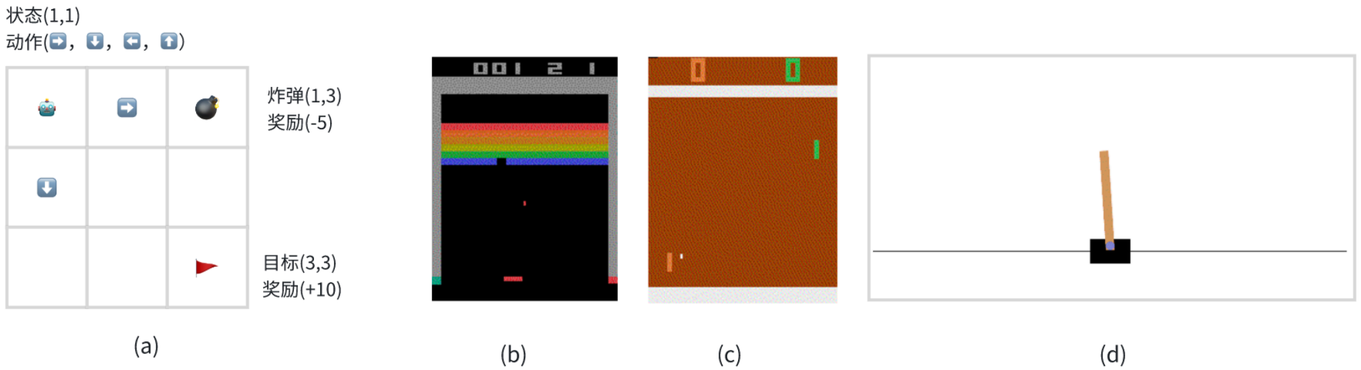


图 7.2　强化学习环境模型

(a)

网格世界：机器人移动到红旗位置，路径奖励和最大。

(b)

打砖块：消灭所有砖块。

(c)

乒乓球：动态对抗对手。

(d)

车杆：保持杆平衡更长时间。

图 (b)–(d) 的环境实现在 Gymnasium 主页（OpenAI Gym 的维护版本，openai/gym）可直接调用，车杆对应 `CartPole-v1`、打砖块对应 `ALE/Breakout-v5`。

智能体与环境

智能体

执行任务的实体

基于当前状态 $s \in \mathcal{S}$，按策略 π 决策动作 $a \in \mathcal{A}$，与环境交互后获得奖励。策略定义了智能体的决策规则。

环境

由五个要素定义

状态集 $\mathcal{S}$、动作集 $\mathcal{A}$、状态转移概率 $p(s' \mid s, a)$、奖励函数 $r(s, a)$ 与折扣因子 γ。

已知环境

$r(s, a)$ 与 $p(s' \mid s, a)$ 均明确，可用动态规划或启发式搜索求最优策略。

网格世界的转移是确定的：目标 $+10$，炸弹 $(1, 3)$ 得 -5，其他移动 -1；每个时刻只能上下左右移动一格。

未知环境

奖励函数或状态转移概率未知，需按策略交互试错学习。

打砖块、乒乓球等 Atari 游戏与车杆物理模型都属于此类，挑战更大。

状态与动作

状态 S_t

- 环境的状态集 $\mathcal{S}$ 包含所有可能的状态。
- 可以是离散的：网格环境的二维坐标，共 9 个状态。
- 也可以是连续的：车杆环境是 4 维向量（车位置、车速度、杆角度、杆角速度）。
- 连续状态数量无限，难以用表格完整记录。

动作 A_t

- 环境的动作集 $\mathcal{A}$ 包含所有可能的动作。
- 动作表示智能体的决策，状态则为决策提供信息依据。
- 离散动作：网格环境 $\{A_{左}, A_{右}, A_{上}, A_{下}\}$，车杆 $\{A_{左}, A_{右}\}$。
- 连续动作：驾驶场景的三维动作（油门、刹车、方向盘转角），可离散化处理。

> **对照** 状态是环境的输出、决策的输入；动作是策略的输出、环境的输入。二者维度的离散或连续，直接决定后续用表格型方法还是函数近似方法。

状态转移概率

$$p(s', r \mid s, a) = \Pr\left[S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a\right] \tag{7-3}$$

$$p(s' \mid s, a) = \Pr\left[S_{t+1} = s' \mid S_t = s, A_t = a\right] = \sum_{r \in \mathcal{R}} p(s', r \mid s, a) \tag{7-4}$$

状态转移由四元组 (s, a, s', r) 描述：在状态 s 选择动作 a 后，环境依概率 $p(s', r \mid s, a)$ 转移到 s' 并产生奖励 r；对 r 求和即得只含状态的转移概率。

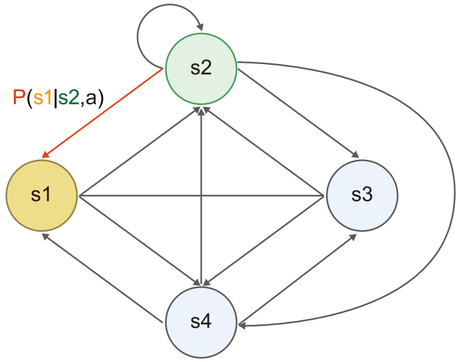


图 7.3　状态转移示意图

s_2 执行 a 后，可能转移到 s_1、s_3，也可能留在 s_2。

奖励函数

$$r(s,a)=\mathbb{E}\left[R_{t+1}\mid S_t=s,A_t=a\right]=\sum_{r\in\mathcal{R}}r\sum_{s'\in\mathcal{S}}p(s',r\mid s,a) \tag{7-5}$$

$r(s,a)$ 为标量，表示在状态 s 执行动作 a 后的期望即时奖励；时刻 t 获得的奖励记为随机变量 R_t。

- 奖励是环境给智能体的唯一评价信号，策略优化的目标由它定义。
- 同一动作在不同状态下可能获得不同奖励，因此奖励写成 (s,a) 的函数。
- 状态转移存在随机性时，对 s' 与 r 求期望，得到与转移无关的确定性标量。

奖励与回报不同：奖励是单步的即时信号，回报（7.1.3 节末）是折扣累积后的长期量。价值函数估计的正是回报的期望。

依据 《大语言模型：训练、精调与对齐》第 7.1.3 节（式 (7-5)）

式 (7-3)–(7-5) 的数值实例

图 7.2(a) 的网格：状态 $s=(2,2)$、动作 $a=$ 右，打滑 10%。

s'	r	$p(s', r \mid s, a)$
(2,3)	−1	0.80
(1,2)	−1	0.10
(3,2)	−1	0.10

三项之和为 1：主方向 0.8、两个垂直方向各 0.1。奖励只由落点决定，因此每个后继只对应一个 r。

$$p(s' \mid s,a)=\sum_{r} p(s', r \mid s, a)$$

$$p((2,3) \mid s,a)=0.80$$

式 (7-4)：对 r 求和，三种后继各得 0.80 / 0.10 / 0.10。

$$\begin{aligned} r(s,a) &= \sum_{r} r \sum_{s'} p(s', r \mid s, a) \\ &= 1.00 \times (-1) \\ &= -1.00 \end{aligned}$$

式 (7-5)：奖励与落点无关地求和，得到一个确定性标量。

对照 同一套计算在 $s=(1,2)$ 向右时得到 $r=-4.20$：有 0.8 的概率直接撞进炸弹 $(1,3)$，把 -5 折进来。

代码 token-to-llm/src/rl/transition.py；式 (7-3)–(7-5)

幕

$$\tau = (S_1, A_1, R_2, S_2, A_2, R_3, \ldots, S_T) \quad (7\text{-}6)$$

幕（episode，或称轨迹）由离散时序 $t = 1, 2, \ldots, T$ 的状态、动作与奖励序列构成；S_T 为终止状态，R_{t+1} 表示执行 A_t 后获得的奖励。

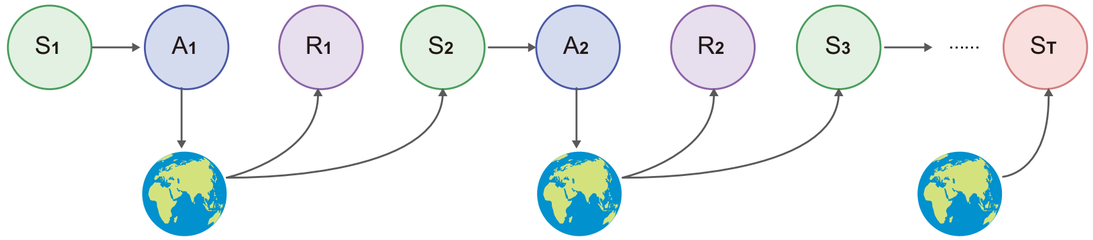


图 7.4　幕数据示例

- 奖励的下标比动作大 1：R_{t+1} 是执行 A_t 之后环境返回的信号。
- 分幕任务在 S_T 结束；连续任务没有终止状态，需要折扣因子保证回报收敛。

图 《大语言模型：训练、精调与对齐》图 7.4；式 (7-6)

回报

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \tag{7-7}$$

折扣因子 $\gamma \in [0,1]$ 调节未来奖励的权重（常取 $\gamma = 0.9$）；第 k 项奖励的系数为 γ^k，与求和式的下标 $t+k+1$ 一致。

作用一

反映未来的不确定性：越远的奖励越不可靠，权重越低。

作用二

保证无限时域下回报收敛：$\gamma < 1$ 时几何级数有界。

作用三

调节智能体的短视程度：$\gamma \to 0$ 只看即时奖励，$\gamma \to 1$ 更重视长期。

回报是对一条具体轨迹算出的标量；价值函数则是对所有可能轨迹取期望。

依据 《大语言模型：训练、精调与对齐》第 7.1.3 节（式 (7-7)）

回报的递归关系

$$G_t = R_{t+1} + \gamma\left(R_{t+2} + \gamma R_{t+3} + \cdots\right) \tag{7-8}$$

$$G_t = R_{t+1} + \gamma G_{t+1} \tag{7-9}$$

把式 (7-7) 中从 R_{t+2} 起的部分提出公因子 γ，括号内正是 G_{t+1} 的定义。

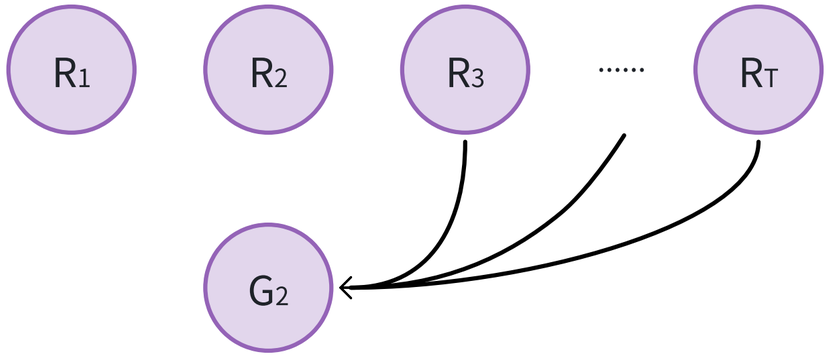


图 7.5　回报计算示意图

要点　式 (7-9) 是全部价值型算法的起点：当前回报 = 即时奖励 + 折扣后的下一回报。贝尔曼方程、时序差分与自举都建立在这一递推之上。

策略

策略 π 定义了在状态 s 下选择动作的规则，可通过学习或搜索得到，目标是最大化长期累积奖励。

确定性策略

$\pi(s) = a$：在状态 s 下直接输出确定的动作 a。

实现简单、执行稳定；但若不加探索，容易漏掉更优动作。

随机性策略

$\pi(a \mid s) = \Pr\{A_t = a \mid S_t = s\}$：给出在状态 s 下选择动作 a 的概率。

记 $\pi(\cdot \mid s)$ 为状态 s 下动作的概率分布，天然带有探索性。

状态 s

环境给出

→

策略 π

确定性映射或概率分布

→

动作 a

作用于环境

依据 《大语言模型：训练、精调与对齐》第 7.1.4 节（策略）

状态价值与动作价值

$$v_\pi(s) = \mathbb{E}_\pi\left[G_t \mid S_t = s\right] \tag{7-10}$$

状态价值函数：从状态 s 出发，在策略 π 下执行动作的累积奖励预测。多幕任务中统计全局状态价值，综合不同幕中 $S_t = s$ 的回报 G_t 的期望。

$$q_\pi(s,a) = \mathbb{E}_\pi\left[G_t \mid S_t = s, A_t = a\right] \tag{7-11}$$

动作价值函数：在状态 s 下执行动作 a 后，继续按策略 π 行动的累积奖励预测。

- $v_\pi(s)$ 评价“处在某个状态有多好”，$q_\pi(s,a)$ 评价“在某个状态做某个动作有多好”。
- 有了 $q_\pi(s,a)$，可直接用 $\arg\max_a q_\pi(s,a)$ 决策动作 a，无需知道环境模型。

依据 《大语言模型：训练、精调与对齐》第 7.1.4 节（式 (7-10)、(7-11)）

最优价值函数与最优策略

$$v_*(s) = \max_\pi v_\pi(s) \tag{7-12}$$

$$q_*(s,a) = \max_\pi q_\pi(s,a) \tag{7-13}$$

$$\pi_* = \arg\max_\pi v_\pi(s) \tag{7-14}$$

$$\pi_* = \arg\max_\pi q_\pi(s,a) \tag{7-15}$$

强化学习的目标是求解最优策略 π_*，以最大化长期累积奖励；最优策略达到最优价值：$v_{\pi_*}(s) = v_*(s)$，$q_{\pi_*}(s,a) = q_*(s,a)$。

- 最优策略通常需要极大计算资源，难以穷举所有状态转移。
- 小规模状态/动作集（表格型任务）可用动态规划求解；未知环境则依赖函数近似。
- 贝尔曼最优方程刻画最优价值函数必须满足的递归性质，见 7.1.5 节。

依据 《大语言模型：训练、精调与对齐》第 7.1.4 节（式 (7-12)–(7-15)）

贝尔曼方程

贝尔曼方程描述价值函数的递归结构，是计算、近似和学习价值函数的基础。先展开状态价值：

$$
\begin{aligned}
v_\pi(s) &= \mathbb{E}_\pi\left[G_t \mid S_t = s\right] \\
&= \sum_{a \in \mathcal{A}} \pi(a \mid s) \cdot \mathbb{E}_\pi\left[G_t \mid S_t = s, A_t = a\right] \\
&= \sum_{a \in \mathcal{A}} \pi(a \mid s) \cdot q_\pi(s, a)
\end{aligned}
$$

第一步按定义展开；第二步用全期望公式，对动作 a 按策略概率 $\pi(a \mid s)$ 加权；第三步代回动作价值函数的定义。

结论：状态价值等于该状态下各动作价值的策略加权平均。反过来，动作价值可由即时奖励与下一状态价值表示，这正是下面两页的推导。

第一步

按定义展开为对回报的期望，$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$ 。

第二步

用全期望公式对动作分层，条件中多出 $A_t = a$，外层按 $\pi(a \mid s)$ 加权。

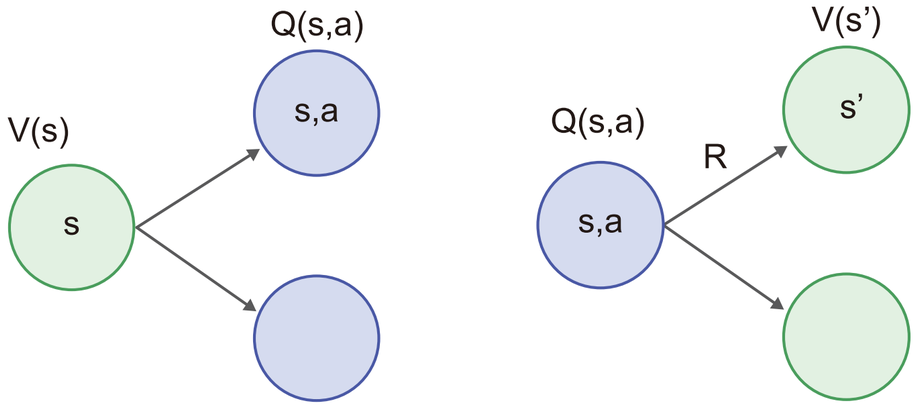
动作价值的展开

$$
\begin{aligned}
q_\pi(s,a) &= \mathbb{E}_\pi\left[G_t \mid S_t=s, A_t=a\right] \\
&= \mathbb{E}_\pi\left[\sum_{k=0}^{\infty}\gamma^k R_{t+k+1} \mid S_t=s, A_t=a\right] \\
&= \mathbb{E}_\pi\left[R_{t+1}+\gamma\sum_{k=0}^{\infty}\gamma^k R_{t+k+2} \mid S_t=s, A_t=a\right] \\
&= \mathbb{E}_\pi\left[R_{t+1} \mid S_t=s, A_t=a\right] + \gamma\,\mathbb{E}_\pi\left[G_{t+1} \mid S_t=s, A_t=a\right] \\
&= r(s,a) + \gamma\sum_{s'\in\mathcal{S}} p(s' \mid s,a)\cdot \mathbb{E}_\pi\left[G_{t+1} \mid S_{t+1}=s'\right] \\
&= r(s,a) + \gamma\sum_{s'\in\mathcal{S}} p(s' \mid s,a)\, v_\pi(s')
\end{aligned}
$$

① 回报写成级数、取出 $k=0$ 项；② 按期望的线性拆项，第一项即 $r(s,a)$，第二项对 s' 展开并用马尔可夫性质换条件；③ 按式 (7-10) 识别为 $v_\pi(s')$。结果 $q_\pi(s,a)=r(s,a)+\gamma\sum_{s'}p(s' \mid s,a)v_\pi(s')$ 与 $v_\pi(s)=\sum_a \pi(a \mid s)q_\pi(s,a)$ 联立即得贝尔曼期望方程。

依据 《大语言模型：训练、精调与对齐》第 7.1.5 节（动作价值的展开）

状态价值与动作价值的转换



(a) 状态价值　　(b) 动作价值

图 7.6　贝尔曼 QV 转化

- $(s) \to (s,a)$：状态价值按策略对动作加权，$v_\pi(s) = \sum_a \pi(a \mid s) q_\pi(s,a)$。
- $(s,a) \to (s')$：动作价值经奖励与转移概率回连到下一状态，$q_\pi(s,a) = r(s,a) + \gamma \sum_{s'} p(s' \mid s,a) v_\pi(s')$。
- 两次代入即得贝尔曼期望方程的两种写法（下页）。

贝尔曼期望方程

$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s)\, q_\pi(s, a) \tag{7-16}$$

$$q_\pi(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' \mid s, a)\, v_\pi(s') \tag{7-17}$$

$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) \left(r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' \mid s, a)\, v_\pi(s') \right) \tag{7-18}$$

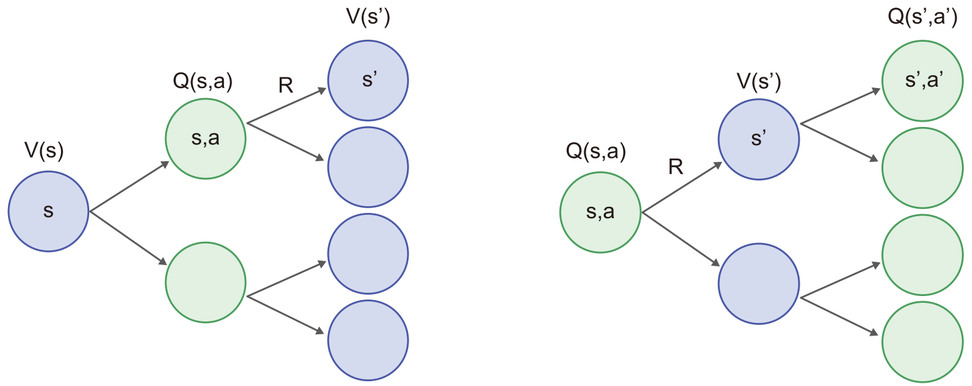
$$q_\pi(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' \mid s, a) \sum_{a' \in \mathcal{A}} \pi(a' \mid s')\, q_\pi(s', a') \tag{7-19}$$

式 (7-16) 把 (7-17) 代入；式 (7-19) 把 (7-16) 代入 (7-17)。$q_\pi(s, a)$ 可由下一状态动作 (s', a') 的 $q_\pi(s', a')$ 回溯计算，$v_\pi(s)$ 同理。

- 四个方程只含 v_π 与 q_π 两个未知函数，给定策略 π 后是线性方程组。
- “期望”来自两处：动作按 π 抽样，下一状态按 p 抽样。

依据 《大语言模型：训练、精调与对齐》第 7.1.5 节（式 (7-16)–(7-19)）

贝尔曼方程的一步回溯



(a) 状态价值　　(b) 动作价值

图 7.7　贝尔曼期望方程示意图

- (a) 状态 s 的价值经动作与奖励，连到下一状态 s' 的价值。
- (b) 动作 (s, a) 的价值经奖励连到 s'，再经策略展开为 (s', a') 的动作价值。
- 两条回溯路径分别对应式 (7-18) 与式 (7-19) 的嵌套求和。

贝尔曼最优方程

$$v_*(s) = \max_{a \in \mathcal{A}} q_*(s,a) \tag{7-20}$$

$$q_*(s,a) = r(s,a) + \gamma \sum_{s' \in \mathcal{S}} p(s' \mid s,a)\, v_*(s') \tag{7-21}$$

$$v_*(s) = \max_{a \in \mathcal{A}} \left(r(s,a) + \gamma \sum_{s' \in \mathcal{S}} p(s' \mid s,a)\, v_*(s') \right) \tag{7-22}$$

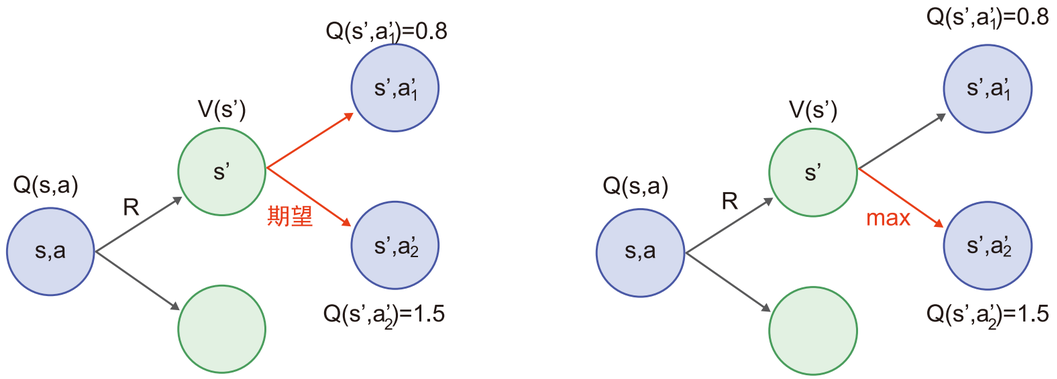
$$q_*(s,a) = r(s,a) + \gamma \sum_{s' \in \mathcal{S}} p(s' \mid s,a) \max_{a' \in \mathcal{A}} q_*(s',a') \tag{7-23}$$

与期望方程的唯一差别：对动作不再按 π 加权，而是取最大值。最优策略 $a_* \sim \arg\max_a q(s,a)$ 可直接给出最大回报，无需依赖特定策略。

当拥有完整环境模型（已知 $p(s' \mid s,a)$ 与 $r(s,a)$）时，可用动态规划求解最优价值（7.3 节）。实际中转移概率与奖励通常未知，于是转向采样方法（7.4、7.5）。

依据 《大语言模型：训练、精调与对齐》第 7.1.5 节（式 (7-20)–(7-23)）

期望方程与最优方程的对比



(a) 期望　　(b) 最优

图 7.8　贝尔曼最优方程示意图

- (a) 对所有下一动作 $(s', a'_1), (s', a'_2)$ 按概率加权，得到期望价值。
- (b) 直接取 $Q(s', a')$ 的最大值，对应贪心动作选择，无需对所有动作积分。
- 核心特性：$v_*(s)$ 已包含未来最优回报的期望；基于 $q_*(s, a)$ 用 $\arg\max_a q_\pi(s, a)$ 选动作更直接，无需额外搜索。

贝尔曼方程的作用

结构

递归分解

把复杂的长远问题转化为“当前一步 + 未来多步”，价值函数的计算由此可迭代。

作用

从抽象到计算

把 MDP 的抽象定义转化为具体的计算目标，即价值函数的最大化。

地位

价值方法的基石

可精确求解、迭代估计和函数近似；基于价值的强化学习方法都以它为基础。

直接计算所有未来路径的奖励总和几乎不可行：时间步可能无限多，状态空间可能非常庞大。贝尔曼方程把这一步不可行的求和，换成了一次只依赖相邻状态的回溯。

下一节先给出方法分类（7.2），再按“模型已知 → 模型未知”的顺序展开：动态规划（7.3）、蒙特卡洛（7.4）、时序差分（7.5）、策略梯度（7.6）与演员-评论家（7.7）。

7.2

强化学习方法

本节内容

7.2　强化学习方法分类

按“学什么”划分三大类：学价值函数再据此选动作、直接学策略、以及把两者结合的演员-评论家。

强化学习方法分类

本书聚焦适用于未知环境的强化学习方法，分为三类。划分依据是“算法最终学到什么、又靠什么选动作”。

一 · 基于价值函数

学价值，再选动作

动态规划、蒙特卡洛方法、时序差分方法。估计 v_π 或 q_π，再用 $\arg\max_a q(s,a)$ 决策。

二 · 基于策略梯度

直接优化策略

对参数化策略 $\pi_\theta(a \mid s)$ 求梯度并做梯度上升，重点是策略梯度的数学推导。

三 · 演员-评论家

价值与策略融合

评论家估计价值、演员更新策略，如演员-评论家算法、TRPO 与 PPO。

环境已知

动态规划

环境未知

蒙特卡洛 / 时序差分

状态连续

策略梯度 / 演员-评论家

表 7.1 汇总本章涉及的全部方法及其特性；7.5.7 节总结基于价值的方法，7.7.4 节总结基于策略的方法。

表 7.1　强化学习方法分类及特性

类型	具体方法	说明
1. 动态规划	值迭代 / 策略迭代	迭代估计状态价值函数。
	广义策略迭代	在满足条件下收敛至最优策略。
2. 蒙特卡洛方法	状态价值估计	通过完整轨迹估计 $V(s)$。
	动作价值估计	估计动作值函数 $Q(s,a)$。
	同轨策略控制	基于当前策略采样更新。
3. 时序差分学习	表格型学习	适用于非分幕任务。
	SARSA	同轨策略，更新依赖 (S, A, R, S', A')。
	Q 学习	经典离轨策略算法。
	期望 SARSA	基于期望 Q 值更新，降低方差。
	DQN	神经网络扩展至连续状态空间。
	多步时序差分	结合蒙特卡洛与 TD 优点估计价值。

表 7.1 上半部分：基于价值函数的三类方法。同类型内的方法共用左侧的类型标签。

表 7.1　强化学习方法分类及特性（续）

类型	具体方法	说明
4. 策略梯度	策略梯度理论	直接优化策略。
	REINFORCE	蒙特卡洛策略梯度方法。
	离轨策略 REINFORCE	使用重要性采样的变体。
5. 演员 - 评论家	演员 - 评论家框架	演员更新策略，评论家评估价值。
	优势演员 - 评论家（A2C）	利用优势函数 $A(s,a)$ 减少方差。
	TRPO	KL 散度约束保证稳定性。
	PPO	目标函数裁剪简化约束。

表 7.1 下半部分：策略梯度与演员-评论家。五类方法依次在 7.3–7.7 展开。

基于价值与基于策略的方法

表 7.1 的五类方法，在本书中归为两条脉络来总结。理解这条分界，才能看清后面每一节在解决什么问题。

基于价值函数

动态规划、蒙特卡洛方法、时序差分学习，统一在 7.5.7 节总结。

先估计价值，再由价值导出策略；策略通常是隐式的（对 Q 取 $\arg\max$）。

难点：连续状态空间下表格无法存储，需要 DQN 之类的函数近似。

基于策略梯度

策略梯度与演员-评论家，统一在 7.7.4 节总结。

直接对策略参数求梯度；价值函数作为基线或评论家，用于降低方差。

难点：梯度估计的方差与每次更新的步长控制，TRPO 与 PPO 正是为此设计。

> **共同点** 两条脉络都建立在贝尔曼方程的递归结构上：价值方法直接用回溯更新价值，策略方法则把状态转移概率内化到稳态分布中，从而绕开对环境模型的建模。

依据 《大语言模型：训练、精调与对齐》第 7.2 节（表 7.1 后的归属说明）

7.3

动态规划

环境模型已知时，直接按贝尔曼方程迭代：先评估一个策略值多少，再据此改进策略，如此往复。

本节内容

7.3 动态规划

7.3.1 小结

动态规划方法

当环境模型已知时，可根据贝尔曼方程用动态规划（DP）评估价值函数并优化策略。在模型已知且计算资源无限的理想条件下，DP 能收敛到最优价值函数与最优策略。

已知模型

$p(s' \mid s, a)$ 与 $r(s, a)$

→

策略评估

迭代求 V_π

→

策略改进

贪心得到 π'

→

收敛

$\pi_* \to v_*$

- DP 通过交替进行**策略评估**与**策略改进**求解最优策略。
- 状态空间 $\mathcal{S}$ 与动作空间 $\mathcal{A}$ 规模过大时，DP 计算开销高。
- 实际中常改用非 DP 方法（蒙特卡洛法或时序差分学习）近似最优价值函数与策略。

依据 《大语言模型：训练、精调与对齐》第 7.3 节

策略评估

策略评估解决“预测”问题：若智能体始终遵循策略 π，它在状态 s 的期望累积回报是多少？给定 π 后，用大小为 $|\mathcal{S}|$ 的表格存 $V_\pi(s)$，迭代更新。

$$
\begin{aligned}
V_\pi(s) &= \mathbb{E}_\pi\left[G_t \mid S_t = s\right] && (7\text{-}24)\\
&= \mathbb{E}_\pi\left[R_{t+1} + \gamma G_{t+1} \mid S_t = s\right] && (7\text{-}25)\\
&= \mathbb{E}_\pi\left[R_{t+1} + \gamma V_\pi(S_{t+1}) \mid S_t = s\right] && (7\text{-}26)\\
&= \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a)\left(r + \gamma V_\pi(s')\right) && (7\text{-}27)
\end{aligned}
$$

式 (7-25) 用回报的递归式 (7-9)；式 (7-26) 用 V_π 替换下一回报的期望（自举）；式 (7-27) 把对动作与对 (s', r) 的期望都写成加权求和。

四步的作用各不相同：定义 → 递归 → 自举 → 加权求和。第 3 步最关键：把“未来所有奖励的期望”换成“下一状态的价值”，此后才能用表格迭代求解。

迭代策略评估

$$V_\pi(s) \leftarrow \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a)\,(r + \gamma V_\pi(s')) \tag{7-28}$$

用旧表算出右端，写回 $V_\pi(s)$。智能体按 $\pi(a \mid s)$ 选动作，环境按 $p(s', r \mid s, a)$ 转移到 s' 并返回 r；更新时对全部可能的 a 与 (s', r) 加权平均。

终止条件：$\max_{s \in \mathcal{S}} |V_{\text{old}}(s) - V_{\text{new}}(s)| < \theta$，其中 $\theta > 0$ 为预设阈值。实现时可维护新旧两个数组（新旧值分离），也可用单数组就地更新。该方法同样可扩展到动作价值 $Q(s, a)$ 的迭代评估。

初始化

$V(s)$ 任意，$V(S_T) = 0$

→

逐状态更新

式 (7-28)

→

求最大变化

$\Delta = \max |v - V(s)|$

→

判断

$\Delta < \theta$ 则停

θ 越小，评估越精确、迭代次数越多；实际中常在精度与耗时之间取折衷。新旧数组分离便于并行，单数组就地更新收敛更快。

算法 1　迭代策略评估算法

算法 1 迭代策略评估算法

输入： 待评估策略 π，状态转移函数 $p(s', r|s, a)$，折扣因子 γ，阈值 $\theta > 0$
输出： 状态价值函数 $V_\pi(s)$

1: 初始化 $V(s)$ 对所有 $s \in \mathcal{S}$，其中终止状态 S_T 满足 $V(S_T) = 0$
2: **repeat**
3: $\quad \Delta \leftarrow 0$
4: $\quad$ **for** 每个状态 $s \in \mathcal{S}$ **do**
5: $\quad\quad v \leftarrow V(s)$ ▷ 保存旧值
6: $\quad\quad V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a)\,(r + \gamma V(s'))$
7: $\quad\quad \Delta \leftarrow \max(\Delta, |v - V(s)|)$
8: $\quad$ **end for**
9: **until** $\Delta < \theta$

以网格环境为例：逐状态按式 (7-28) 更新 $V_\pi(s)$；第 5 步保存旧值，第 7 步用它衡量本轮更新幅度。

策略改进

得到 $V_\pi(s)$ 后，策略改进的目标是寻找更优策略 π'。对每个状态遍历所有可行动作，选使价值最大的动作。

$$\pi'(s) = \arg\max_{a \in \mathcal{A}} \sum_{s'} p(s' \mid s, a)\, V(s')$$

这是策略改进的直观形式：用状态价值 $V(s')$ 乘以转移概率，比较各动作的期望效果，取最大者。

环境存在随机状态转移时（$p(s' \mid s, a) < 1$），需考虑所有可能转移路径的期望价值。动态规划的标准设定中环境转移通常是确定性的，例如 $p(s' \mid s, a) = 1$。使用动作价值 $q_\pi(s, a)$ 的优势是**可直接选择最优动作**，其更新公式由贝尔曼方程导出：

$$q_\pi(s, a) = \mathbb{E}\left[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = a\right] \tag{7-29}$$

$$= \sum_{s', r} p(s', r \mid s, a)\left(r + \gamma v_\pi(s')\right) \tag{7-30}$$

$$\pi'(s) = \arg\max_{a \in \mathcal{A}} q_\pi(s, a) \tag{7-31}$$

式 (7-31) 生成改进策略 π'；新策略满足策略改进定理，即 $\forall s : v_{\pi'}(s) \geqslant v_\pi(s)$。

状态价值与动作价值的开销

用状态价值 $V_\pi(s)$

存储 $|\mathcal{S}|$ 个值，空间最省。

改进策略时要对每个动作重新做一次转移概率加权求和，才能比较动作优劣。

适合转移确定、动作数较少的网格类任务。

用动作价值 $q_\pi(s,a)$

存储 $|\mathcal{S}| \times |\mathcal{A}|$ 个值，空间开销为状态价值函数的 $|\mathcal{A}|$ 倍。

改进时直接取 $\arg\max_a q_\pi(s,a)$，无需再做期望，决策更直接。

后续的 SARSA、Q 学习都沿用动作价值这一表示。

> **选择** 动态规划在网格环境中，动作价值函数的空间开销是状态价值的 $|\mathcal{A}|$ 倍；换来的是策略改进时无需遍历转移概率。这是“存储换计算”的典型权衡。

为什么用 V 比较动作更麻烦

由 V：$\pi'(s) = \arg\max_a \sum_{s'} p(s' \mid s,a)\big[r(s,a) + \gamma V(s')\big]$

由 Q：$\pi'(s) = \arg\max_a Q(s,a)$

左边要多乘一层环境模型 $p(s' \mid s,a)$ 与 $r(s,a)$；右边只是一个查表取最大。

量	条目数	来源
$p(s' \mid s,a)$	324	须估计
$r(s,a)$	36	须估计
$V(s)$	9	可由样本估
$Q(s,a)$	36	可由样本估

实测：7 个非终止状态、4 个动作，场景取 wind（主方向 0.8，0.2 概率多下滑一格）。

12

由 V 每状态改进所需乘加

3

由 Q 每状态改进所需比较

转移模型稍有偏差，由 V 选出的动作就错。把 wind 误设为确定性转移后，7 个状态里有 **3 个**选错动作（(1,1)、(2,1)、(2,2)），平均 v 从 $+7.934$ 掉到 $+7.666$。

> **结论** V 缺动作维度，比较动作时只能“补一层模型”；Q 把动作写进下标，贪心改进不必知道环境怎么转移。

代码 token-to-llm/src/rl/why_v_is_harder.py

广义策略迭代

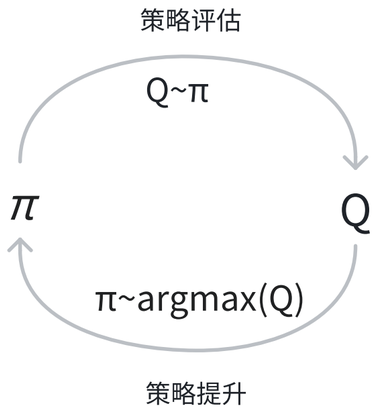


图 7.9　广义策略迭代

$$\pi_0 \xrightarrow{\text{评估}} v_{\pi_0} \xrightarrow{\text{改进}} \pi_1 \xrightarrow{\text{评估}} v_{\pi_1} \xrightarrow{\text{改进}} \cdots \xrightarrow{\text{改进}} \pi_* \xrightarrow{\text{评估}} v_* \tag{7-32}$$

广义策略迭代（GPI）交替进行策略评估与策略改进：一次评估把当前策略的价值估准，一次改进把策略推向更优，直到收敛到 π_* 与 v_*。

- 价值估计 v_{π_i} 逐步逼近当前策略 π_i 的真实价值，同时策略持续改进。
- GPI 不仅是动态规划的理论框架，也支撑蒙特卡洛法与时序差分法。
- 那些方法只能近似求解，但 GPI 框架仍保证其收敛到最优解附近。

图 《大语言模型：训练、精调与对齐》图 7.9；式 (7-32)

策略改进定理

$$\pi'(s) = \arg\max_{a \in \mathcal{A}} q_\pi(s, a) \tag{7-33}$$

GPI 中的策略改进以贪心操作实现：对每个状态，取动作价值最大的动作作为新策略。

$$\begin{aligned} q_\pi\left(s, \pi'(s)\right) &= \max_{a \in \mathcal{A}} q_\pi(s, a) && (7\text{-}34) \\ &\geqslant q_\pi\left(s, \pi(s)\right) && (7\text{-}35) \\ &\geqslant v_\pi(s) && (7\text{-}36) \end{aligned}$$

式 (7-34) 由贪心选择直接得到；式 (7-35) 因为 $\pi(s)$ 也是可选项之一；式 (7-36) 因为按 π 对动作取期望不会超过其中最大的那个（$v_\pi(s) = \sum_a \pi(a \mid s) q_\pi(s, a) \leqslant \max_a q_\pi(s, a)$）。

三步连起来即 $q_\pi(s, \pi'(s)) \geqslant v_\pi(s)$，说明新策略在每个状态都不比旧策略差；反复应用即得策略改进定理 $\forall s：v_{\pi'}(s) \geqslant v_\pi(s)$。

依据 《大语言模型：训练、精调与对齐》第 7.3 节（式 (7-33)–(7-36)）

动态规划方法的总结

机制

递归求值

状态价值由即时奖励 r 与折扣后的下一状态价值 $\gamma v_\pi(s')$ 构成，这一迭代结构适用于所有状态的价值评估。

目的

评估为改进服务

策略评估的目标是为策略改进提供基础：通过状态价值函数推导出更优策略。

边界

依赖环境模型

环境未知时贝尔曼方程无法直接求解，需依赖经验数据估计期望回报，或采用模型学习方法。

动态规划把贝尔曼方程当成一份可以直接执行的“作业单”：只要知道 $p(s' \mid s, a)$ 与 $r(s, a)$，就能对每个状态反复代入、反复改进。

下一节起，模型不再已知。7.4 的蒙特卡洛用完整幕的回报替代期望，7.5 的时序差分进一步只用一步奖励加自举。

7.4

蒙特卡洛方法

不再需要状态转移概率：采样足够多的完整幕，用状态的平均回报直接估计它的价值，再据此改进策略。

本节内容

7.4.1 蒙特卡洛价值评估

7.4.2 试探性出发蒙特卡洛控制

7.4.3 非试探性出发蒙特卡洛控制

蒙特卡洛方法的基本思路

动态规划中的广义策略迭代需要已知状态转移概率。未知环境中，状态价值的估计可以通过回报的期望获得，即 $v_\pi(s) = \mathbb{E}\left[G_t \mid S_t = s\right]$。蒙特卡洛（MC）用采样实现这个期望。

从 S_1 出发

按策略 π 采样

→

N 幕完整数据

直到终止状态 S_T

→

算每幕的 G_t

从后向前折扣累加

→

求平均

作为 $v_\pi(s)$ 的估计

- MC 直接从幕数据学习，**无需对环境建模**。
- 为估计回报 G_t，必须采样完整的幕序列 $S_1, A_1, R_1, \ldots, S_T, A_T, R_T$，包含终止状态 S_T。
- “完整”是 MC 与后续时序差分方法最根本的分界：MC 要等到一幕结束才能更新。

依据 《大语言模型：训练、精调与对齐》第 7.4.1 节

折扣回报的平均估计

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1} \tag{7-37}$$

与无限时域的式 (7-7) 相比，求和上限由 ∞ 改为 $T-t-1$：幕在 T 结束，此后的奖励不存在。

$$v_\pi(s) = \frac{\sum_{i=1}^{N} \sum_{t=1}^{T} \mathbb{I}\left[S_t^{(i)} = s\right] G_t^{(i)}}{\sum_{i=1}^{N} \sum_{t=1}^{T} \mathbb{I}\left[S_t^{(i)} = s\right]} \tag{7-38}$$

$\mathbb{I}[\cdot]$ 为二值指示函数，条件满足返回 1，否则返回 0；$S_t^{(i)}$ 是第 i 幕 t 时刻的状态。分子累加所有访问到 s 的回报，分母是访问次数，比值即平均回报。

在一幕数据中，状态 s 可能未被访问、访问一次或多次。式 (7-38) 把每次访问都计入，因此称为**每次访问蒙特卡洛估计**（Every-Visit MC）。

首次访问蒙特卡洛估计

若仅取每幕首次访问状态 s 的回报计算均值，则称为首次访问蒙特卡洛估计（First-Visit MC）。

$$v_\pi(s) = \frac{\sum_{i}^{N}\sum_{t=1}^{T}\mathbb{F}(s,t,i)\,G_t^{(i)}}{\sum_{i}^{N}\sum_{t=1}^{T}\mathbb{F}(s,t,i)} \tag{7-39}$$

$$\mathbb{F}(s,t,i) = \mathbb{I}\left[S_t^{(i)} = s \text{ 且 } \forall \tilde{t} < t,\ S_{\tilde{t}}^{(i)} \neq s\right] \tag{7-40}$$

$\mathbb{F}(s,t,i)$ 是首次访问函数：第 i 幕 t 时刻首次访问状态 s 时取 1，即此前所有时刻 $\tilde{t} < t$ 都没有访问过 s。

两式的结构完全相同，只把指示函数 $\mathbb{I}$ 换成首次访问函数 $\mathbb{F}$：分子分母同步变化，比值仍是“首次访问回报的均值”。

$\mathbb{I}$ 的作用

回答“这一刻是否在 s”：只做一次相等判断。

$\mathbb{F}$ 的作用

回答“这一刻是否是第一次在 s”：还要检查此前所有时刻。

依据 《大语言模型：训练、精调与对齐》第 7.4.1 节（式 (7-39)、(7-40)）

算法 2　首次访问型蒙特卡洛估计算法

算法 2 首次访问型蒙特卡洛估计算法

输入： 策略 π；状态集 $\mathcal{S}$；初始状态价值估计 $V(s)$（任意值）；空列表 $\hat{G}(s)$（$\forall s \in \mathcal{S}$）
输出： 状态价值函数估计 $V(s) \approx v_\pi(s)$

for 幕索引 $i = 1$ 到 N **do**
　生成幕数据：$S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T, S_T$　▷ 终止状态 S_T
　$G \leftarrow 0$　▷ 初始化累积回报
　for $t = T-1, T-2, \ldots, 0$ **do**　▷ 反向遍历时刻
　　$G \leftarrow \gamma G + R_{t+1}$　▷ 更新折扣回报
　　if 状态 S_t 未在 $S_0, S_1, \ldots, S_{t-1}$ 中出现 **then**　▷ 首次访问检查
　　将 G 加入列表 $\hat{G}(S_t)$
　　$V(S_t) \leftarrow \text{average}(\hat{G}(S_t))$　▷ 更新价值估计
　　end if
　end for
end for

第 5 步从后向前递推：每退一步，先给已有回报乘 γ，再加上新的一步奖励，等价于式 (7-37) 的折扣累加。

首次访问与每次访问

首次访问

每个状态只使用其首次访问的回报。

各回报值相互独立，都是价值估计的独立同分布估计，**方差较低**。

适合对方差敏感的场景。

每次访问

一幕中多次访问同一状态时，每次都用其回报更新估计。

样本量更大，但同一状态的多次访问回报往往来自同一条轨迹、彼此相关，**方差可能更高**。

更注重样本效率。

> **权衡** 两种估计在幕数趋于无穷时都收敛到 $v_\pi(s)$；差别在于有限样本下的方差与样本效率。首次访问以丢弃重复访问为代价换取独立性，每次访问则相反。

动作价值的估计

同理，把“状态”换成“状态-动作对”，即得动作价值的首次访问蒙特卡洛估计。

$$q_\pi(s,a)=\frac{\sum_{i=1}^{N}\sum_{t=0}^{T-1}\mathbb{F}(s,a,t,i)\,G_t^{(i)}}{\sum_{i=1}^{N}\sum_{t=0}^{T-1}\mathbb{F}(s,a,t,i)} \tag{7-41}$$

$$\mathbb{F}(s,a,t,i)=\mathbb{I}\left[(S_t^{(i)},A_t^{(i)})=(s,a)\ \text{且}\ \forall \tilde{t}<t,\ (S_{\tilde{t}}^{(i)},A_{\tilde{t}}^{(i)})\neq(s,a)\right] \tag{7-42}$$

首次访问指示函数 $\mathbb{F}(s,a,t,i)$ 表示第 i 幕中状态-动作对 (s,a) 在时刻 t 首次被访问。求和上限为 $T-1$，因为终止状态之后不再有动作。

- 动作价值是控制问题的核心：有了 $q_\pi(s,a)$ 才能对策略做贪心改进。
- 状态价值只回答“站在哪里好”，动作价值还回答“该往哪走”。

依据 《大语言模型：训练、精调与对齐》第 7.4.1 节（式 (7-41)、(7-42)）

蒙特卡洛与动态规划的差别

蒙特卡洛

回报从终止时刻反向计算（回溯）。

对每个状态的价值估计**相互独立**，不依赖其他状态的估计。

通过采样通常获得**次优估计**，但对期望回报无偏。

动态规划

用奖励和下一时刻的价值估计更新当前价值，即**自举**。

需要已知环境模型，可找到全局最优解。

估计之间有依赖，误差会在状态之间传播。

> **要点** “是否自举”与“是否需要完整幕”是划分价值方法的两个坐标：MC 不自举、要完整幕；DP 自举、要模型；下一节的 TD 则自举、且只要一步数据。

逐幕交替的评估与改进

用蒙特卡洛方法得到更优的价值估计后，即可据此改进策略，实现广义策略迭代。与 7.3 节的 GPI 图（图 7.9）相同，只是“评估”改由采样完成。

采样一幕
按当前策略生成完整幕

→

策略评估
用观测回报更新 Q

→

策略改进
在该幕访问过的状态上贪心

→

再采样
用改进后的策略

- 采用逐幕交替的评估-改进流程：每幕结束后先评估，再改进，然后用改进后的策略采样新一幕。
- 试探性出发：从随机状态-动作对 (s, a) 开始采样，保证所有节点都能被访问。

评估

用这一幕的回报更新 $Q(S_t, A_t)$，只统计本幕首次访问的那一对。

改进

在同一遍反向遍历中直接改写 $\pi(S_t)$，无需等整幕评估完再统一改。

算法 3　试探性出发首次访问蒙特卡洛控制

算法 3 试探性出发首次访问蒙特卡洛控制

输入： 初始策略 $\pi(s) \in \mathcal{A}(s)$ $(\forall s \in \mathcal{S})$；动作价值估计 $Q(s,a)$（任意初始化）；空回报列表 $\hat{G}(s,a)$ $(\forall s,a)$
输出： 最优动作价值估计 $Q(s,a) \approx q_*(s,a)$

1: **loop**
2: 　随机选择初始状态 S_0 和动作 A_0 ▷ 试探性出发
3: 　生成幕数据：$S_0, A_0, R_1, S_1, A_1, \ldots, R_T, S_T$
4: 　$G \leftarrow 0$ ▷ 初始化累积回报
5: 　**for** $t = T-1, T-2, \ldots, 0$ **do** ▷ 反向遍历
6: 　　$G \leftarrow \gamma G + R_{t+1}$ ▷ 更新折扣回报
7: 　　**if** 状态-动作对 (S_t, A_t) 未在 $(S_0, A_0), \ldots, (S_{t-1}, A_{t-1})$ 出现 **then**
8: 　　　将 G 加入列表 $\hat{G}(S_t, A_t)$
9: 　　　$Q(S_t, A_t) \leftarrow \text{average}(\hat{G}(S_t, A_t))$ ▷ 更新动作价值
10: 　　　$\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$ ▷ 策略改进
11: 　　**end if**
12: 　**end for**
13: **end loop**

第 10 步把评估与改进放在同一遍反向遍历中完成：一处估计更新后，立即把该状态的策略改为贪心。

ε-贪心策略

试探性出发要求从任意状态-动作对出发，实际场景中往往不可行：机器人无法从任意状态启动；确定性策略下某些状态-动作对可能永远不被访问。

因此需要设计从固定状态出发、却能探索所有状态-动作对的采样方案。ϵ-贪心策略可平衡探索与利用：

$$\pi(a \mid s) = \begin{cases} 1 - \epsilon + \dfrac{\epsilon}{|\mathcal{A}(s)|} & \text{if } a = \arg\max_{a'} Q(s, a') \\ \dfrac{\epsilon}{|\mathcal{A}(s)|} & \text{otherwise} \end{cases} \tag{7-43}$$

以 $1-\epsilon$ 的概率选择当前最优动作（利用），以 ϵ 的概率在全部动作上均匀随机选择（探索）；最优动作本身也分到 $\epsilon/|\mathcal{A}(s)|$ 的份额。

两种情形

$a = \arg\max_{a'} Q(s, a')$ 时概率为 $1 - \epsilon + \epsilon/|\mathcal{A}(s)|$；其余动作各为 $\epsilon/|\mathcal{A}(s)|$。

归一检查

各类概率相加为 $1 - \epsilon + \epsilon = 1$，因此仍是一个合法的概率分布。

ε 的取值

较大的 ϵ，缓慢衰减

探索更充分，几乎所有状态-动作对都能被反复访问。

提高收敛的**可靠性**，但收敛速度变慢。

较小的 ϵ，快速衰减

更快把采样集中在当前最优动作附近。

可能加速收敛，但容易陷入**局部最优**。

> **做法** 实践中常让 ϵ 从较大值开始并逐步衰减：训练初期以探索为主，保证覆盖；后期以利用为主，让策略收敛。

ϵ-贪心是后续 SARSA、Q 学习、DQN 共用的行为策略，7.5 节会反复用到。

7.5

时序差分学习

不必等一幕结束：用“一步奖励 + 下一状态的当前估计”构造 TD 目标，边交互边更新价值。

本节内容

7.5.1 时序差分策略评估

7.5.2 SARSA：时序差分控制

7.5.3 Q 学习：离轨策略的时序差分控制

7.5.4 期望 SARSA

7.5.5 深度 Q 网络：DQN

7.5.6 多步时序差分

7.5.7 基于价值函数的强化学习方法总结

时序差分与蒙特卡洛的比较

时序差分（TD）学习与蒙特卡洛一样是无模型方法。区别在于：MC 必须从完整幕数据学习，要等一幕结束才能更新；TD 可以直接从不完整的幕数据学习，无需等待幕终止。

蒙特卡洛

依赖**真实回报** G_t，必须采样到终止状态 S_T。

回报是实际发生的，估计无偏，但方差较大。

只能用于分幕任务。

时序差分

采用**自举**：用当前价值估计 $V(S_{t+1})$ 构造目标，更新 $V(S_t)$。

只用一步数据即可更新，方差小，但引入偏差。

可用于非分幕的连续任务。

区别 MC 等到最后用真实回报算账，TD 每走一步就先按当前估计“预支”下一状态的价值，再据此修正当前估计。

TD(0) 更新

TD 学习的核心是用 TD 目标 $R_{t+1}+\gamma V(S_{t+1})$ 更新状态价值 $V(S_t)$，α 为步长。

$$V(S_t) \leftarrow (1-\alpha)V(S_t)+\alpha G_t \tag{7-44}$$

$$\leftarrow V(S_t)+\alpha\left(G_t-V(S_t)\right) \tag{7-45}$$

$$\leftarrow V(S_t)+\alpha\left(R_{t+1}+\gamma V(S_{t+1})-V(S_t)\right) \tag{7-46}$$

式 (7-44) 是旧估计与新目标的凸组合；式 (7-45) 把它改写成"旧值 + 步长 × 误差"的增量形式；式 (7-46) 把 MC 的完整回报 G_t 换成单步 TD 目标 $R_{t+1}+\gamma V(S_{t+1})$。

由于只依赖相邻时序的估计，该方法称为 $\mathrm{TD}(0)$。括号内的 $R_{t+1}+\gamma V(S_{t+1})-V(S_t)$ 即 TD 误差，是后续 SARSA、Q 学习与演员-评论家共用的核心量。

式 (7-44)

旧估计与新目标的凸组合，α 越小越保守。

式 (7-45)

改写成"旧值 + 步长 × 误差"，便于逐样本增量实现。

式 (7-46)

把 G_t 换成 TD 目标，只需一步数据即可更新。

依据 《大语言模型：训练、精调与对齐》第 7.5.1 节（式 (7-44)–(7-46)）

算法 4　表格型 TD(0) 策略评估

算法 4 表格型 TD(0) 策略评估

输入： 给定初始策略 π，对所有状态 S，初始价值 $V(S)$, 其中 $V(\text{终止状态}) = 0$，步长 $\alpha \in (0, 1]$, $\epsilon \in [0, 1]$

输出： 最优状态值函数 $V^*(s, a)$

1: **for** 每幕循环 **do**
2:　　初始化 S
3:　　**for** 对幕中的每一步循环 **do**
4:　　$A \leftarrow \pi(S)$
5:　　执行动作 A, 得到环境的奖励 R 和下一个状态 S'
6:　　$V(S) \leftarrow V(S) + \alpha(R + \gamma V(S') - V(S))$
7:　　$S \leftarrow S'$
8:　　**end for**
9:　　直到 S 是终止状态
10: **end for**

第 6 步就是式 (7-46)：每走一步立即更新，不必等整幕结束。这是策略评估算法，学到的是给定策略 π 的价值 $V_\pi(s)$。

TD 目标

在时序差分估计中，TD 目标 $R_{t+1} + \gamma V(S_{t+1})$ 替代了 MC 的完整回报 G_t，从而避免了完整回报的计算。

执行 A_t

观测 R_{t+1} 与 S_{t+1}

→

构造目标

$R_{t+1} + \gamma V(S_{t+1})$

→

算 TD 误差

目标 $- V(S_t)$

→

更新

$V(S_t) \mathrel{+}= \alpha \cdot \delta$

偏差

目标里含估计值 $V(S_{t+1})$，因此 TD 估计是有偏的。

方差

只用一步随机奖励，方差远小于 MC 的整条轨迹回报。

代价

偏差与方差的折衷程度，由后续的多步 TD 与 TD(λ) 显式调节。

与 MC 的对照：MC 用 G_t 作目标，无偏但方差大、必须等到幕终止；TD 用 $R_{t+1} + \gamma V(S_{t+1})$ 作目标，有偏但方差小、每步都能更新。这一取舍是 7.5.6 节多步 TD 的出发点。

依据 《大语言模型：训练、精调与对齐》第 7.5.1 节

SARSA 算法

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right) \tag{7-47}$$

算法名称来自更新所用的五元组 $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$ 的字母。在 S_t 按 π 选 A_t，转移到 S_{t+1} 得 R_{t+1}；若 S_{t+1} 非终止，再按同一策略选 A_{t+1}。

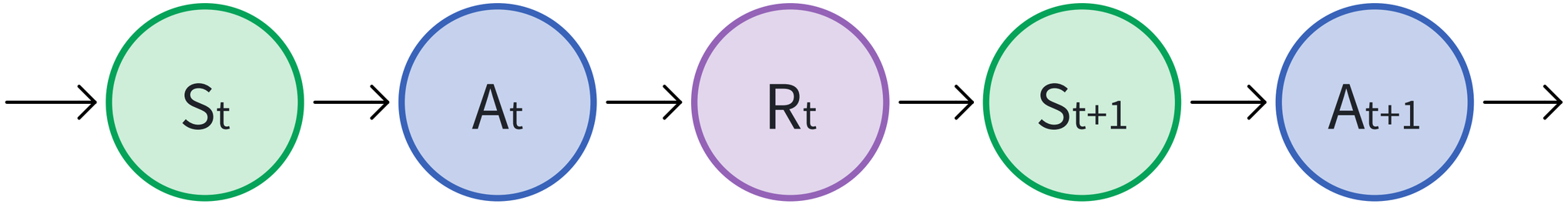


图 7.10　SARSA 五元组

- 与 TD(0) 唯一的区别：被估计与自举的对象从状态价值 V 换成动作价值 Q。
- 当前动作 A_t 与下一动作 A_{t+1} 都由**相同的行为策略**（ϵ-贪心）生成，因此 SARSA 是同轨策略算法。

五元组的读法：(S_t, A_t) 是"现在"，R_{t+1} 是"这一步的反馈"，(S_{t+1}, A_{t+1}) 是"下一步"。更新目标由后三者构成，这正是 TD 目标在动作价值上的写法。

算法 5　SARSA 时序差分控制算法

算法 5 SARSA 时序差分控制算法

输入： 对所有 s, a，任意初始动作价值 $Q(s,a)$, 其中 $Q(S_T, \cdot)$ 终止状态为 0，步长 $\alpha \in (0,1]$, $\epsilon \in [1,0]$，收敛阈值 θ

输出： 最优状态值 $Q^*(s,a) \approx q_*$

1: **for** 每幕循环 **do**
2: 　初始状态 S
3: 　使用从 Q 价值函数得到的策略（如 ϵ -贪心），在 S 处选择动作 A
4: 　**for** 对幕中的每一步循环 **do**
5: 　　执行动作 A, 得到环境的奖励 R 和下一个状态 S'
6: 　　使用从 Q 得到的策略 (例如 ϵ-贪心)，在 S' 处选择 A'
7: 　　$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$
8: 　　$S \leftarrow S'$; $A \leftarrow A'$
9: 　**end for**
10: 　直到 S 是终止状态
11: **end for**

第 3 步与第 6 步都用同一套 ϵ-贪心规则选动作；第 8 步把 (S', A') 滚动为下一步的 (S, A)，避免重复采样。

SARSA 的 GPI 框架

行为策略

ϵ-贪心，从当前 Q 导出

→

五元组

$(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$

→

TD 评估

式 (7-47) 更新 Q

→

策略改进

Q 变了，ϵ-贪心随之变

- SARSA 采用广义策略迭代框架：策略评估用 TD 方法，策略改进通过 ϵ-贪心平衡探索与利用。
- 改进不是显式重写策略表，而是隐含在“Q 更新后重新按 ϵ-贪心抽样”这一循环里。
- 同轨策略的含义：**采样用的策略与正在评估、改进的策略是同一个**。走到哪学到哪，学到的就是自己当前行为方式的价值。

评估

用 TD 目标 $R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$ 更新 $Q(S_t, A_t)$，只回溯一步。

改进

下一轮采样时按更新后的 Q 重新做 ϵ-贪心，策略随之变好。

依据 《大语言模型：训练、精调与对齐》第 7.5.2 节

同轨策略与离轨策略

同轨策略（On-Policy）

用于采样数据序列的策略，与待评估和改进的策略**相同**。

代表算法：SARSA。

学到的是“我这样带探索地走，值多少”。

离轨策略（Off-Policy）

采样策略（行为策略）与优化策略**不同**。

代表算法：Q 学习、DQN。

用独立的行为策略（历史策略或随机策略）探索不同轨迹，增强探索性。

> **动机** 引入离轨策略的主要原因，是让采样可以来自与目标策略不同的轨迹：既能保留探索带来的覆盖，又能直接朝最优策略学习。

Q 学习

Q 学习用 ϵ-贪婪策略采样，得到四元组 $(S_t, A_t, R_{t+1}, S_{t+1})$。其中 A_{t+1} 不由采样策略得到，而是直接取 S_{t+1} 下最大动作价值对应的动作，即 $\max_{a\in\mathcal{A}} Q(S_{t+1}, a)$。

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left(R_{t+1} + \gamma \max_{a\in\mathcal{A}} Q(S_{t+1}, a) - Q(S_t, A_t) \right) \tag{7-48}$$

$Q(S_t, A_t)$ 以最优动作价值估计 $\max_{a\in\mathcal{A}} Q(S_{t+1}, a)$ 为学习目标。相比 SARSA，Q 学习直接优化最优策略的价值函数，具有更优的收敛潜力。

- 离轨策略体现为：采样时用 ϵ-贪心（行为策略），优化目标用贪心（目标策略）。
- 因此能从任意行为策略生成的数据中学习最优策略，可以重用历史数据。

算法 6　Q 学习（离轨策略）下的 TD 控制算法

算法 6 Q-Learning（离轨策略）下的 TD 控制算法

输入： 对所有 s, a，任意初始动作价值 $Q(s,a)$, 其中 $Q(S_T, \cdot)$ 终止状态为 0，步长 $\alpha \in (0,1]$, $\epsilon \in [0,1]$
输出： 最优状态值函数 $Q^*(s,a) \approx q_*$

for 每幕循环 **do**
　初始状态 S
　for 对幕中的每一步循环 **do**
　　使用从 Q 得到的策略 (例如 ϵ-贪心)，在 S 处选择 A
　　执行动作 A, 得到环境的奖励 R 和下一个状态 S'
　　$Q(S,A) \leftarrow Q(S,A) + \alpha(R + \gamma \max_{a \in \mathcal{A}} Q(S',a) - Q(S,A))$
　　$S \leftarrow S'$;
　end for
　直到 S 是终止状态
end for

与算法 5 对比：第 4 步仍用 ϵ-贪心选动作，但第 6 步的目标里出现的不是采样到的 A'，而是 $\max_a Q(S',a)$。

SARSA 与 Q 学习的区别

SARSA

同轨策略：行为策略与目标策略一致。

更新目标为实际采样动作 A_{t+1} 的价值 $Q(S_{t+1}, A_{t+1})$。

收敛至**当前策略**的动作价值函数 q_π。

Q 学习

离轨策略：行为策略与目标策略分离。

更新目标为 $\max_{a \in \mathcal{A}} Q(S_{t+1}, a)$，即 S_{t+1} 下的最优动作价值估计。

直接学习最优策略的动作价值 $Q^* \approx q_*$，并已证明收敛至 q_*。

直观差别 SARSA 用“下一步实际做了什么”来估计，Q 学习用“下一步最好能做什么”来估计。前者更保守，后者更激进。

期望 SARSA

期望 SARSA 结合了 SARSA 与 Q 学习的思想，核心改进是用期望值更新动作价值函数。

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left(R_{t+1} + \gamma \, \mathbb{E}_{A_{t+1} \sim \pi} \left[Q(S_{t+1}, A_{t+1}) \right] - Q(S_t, A_t) \right) \tag{7-49}$$

$$\leftarrow Q(S_t, A_t) + \alpha \left(R_{t+1} + \gamma \sum_{a \in \mathcal{A}} \pi(a \mid S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t) \right) \tag{7-50}$$

式 (7-50) 把期望写成对全部动作的加权求和：权重是策略在 S_{t+1} 上选择各动作的概率。

比较三种目标：SARSA 取**实际采样动作**的 Q，Q 学习取**最大值** $\max_a Q$，期望 SARSA 取**按 π 加权的期望**。前者方差大，中者有最大化偏差，后者居中。

SARSA

$R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$

Q 学习

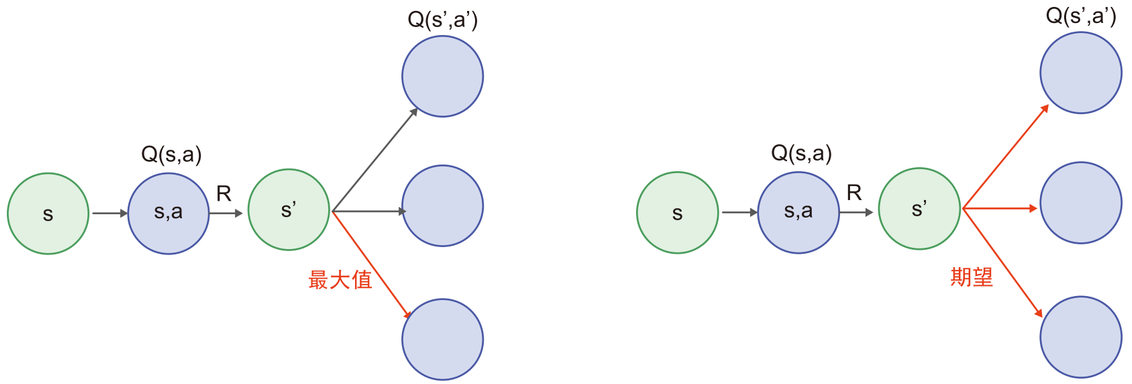
$R_{t+1} + \gamma \max_a Q(S_{t+1}, a)$

期望 SARSA

$R_{t+1} + \gamma \sum_a \pi(a \mid S_{t+1}) Q(S_{t+1}, a)$

依据 《大语言模型：训练、精调与对齐》第 7.5.4 节（式 (7-49)、(7-50)）

三种更新机制的对比



(a) Q 学习　　(b) 期望 SARSA

图 7.11　Q 学习与期望 SARSA 更新机制对比

- (a) 在 s' 的多个 $Q(s', a')$ 中取最大值，作为 $Q(s, a)$ 的更新目标。
- (b) 对 s' 下的各 $Q(s', a')$ 按策略概率取期望，目标更平滑。

图 《大语言模型：训练、精调与对齐》图 7.11；式 (7-48)–(7-50)

期望 SARSA 的特点

- **显著降低方差**：对下一动作取期望，消除了“这一步恰好抽到哪个动作”带来的随机波动，提升学习稳定性。
- **避免最大化偏差**：Q 学习中的 $\max$ 会系统性地高估价值；期望形式的价值估计更接近真实期望。
- **减少过早收敛**：策略不易过早收敛到次优解。

策略归属

期望 SARSA 既可以是同轨策略，也可以是离轨策略，取决于期望所用的 π 是行为策略还是目标策略。

计算代价

由于期望基于当前 Q 函数与策略，其更新目标无偏且方差低于标准 SARSA；动作空间较大时，可用采样近似期望。

三种目标的对照可以写成一句话：SARSA 用“实际走的那一步”，Q 学习用“最好的一步”，期望 SARSA 用“平均的一步”。

何时选它

动作空间不大、希望更新平稳、不愿被单次采样的随机动作带偏时。

何时不用

动作空间极大、期望求和代价过高时，改用采样近似或直接回到 SARSA / Q 学习。

依据 《大语言模型：训练、精调与对齐》第 7.5.4 节

同轨策略与离轨策略的对比

同轨策略

行为策略与目标策略相同，如 SARSA。

在策略迭代框架下（如 ϵ-贪心）通常收敛到最优策略的**近似解**。

需要实时生成交互数据。

离轨策略

行为策略与目标策略不同，如 Q 学习。

优势：能重用历史数据、并行学习多个策略，探索更灵活。

缺点：估计方差较高，收敛速度通常较慢。

> **选择** 能否重用旧数据，是工程上最实际的差别：离线语料充足时倾向离轨方法，在线交互成本低、追求稳定时倾向同轨方法。

Q 网络

状态空间与动作空间非常大或连续时，用一张巨大的表格存 $Q(s,a)$ 需要巨大的存储资源。改用带参数的模型估计动作价值，即价值函数估计。

- 用参数为 θ 的模型计算动作价值，记为 $q(s,a;\theta)$，也称“Q 网络”。
- 输入是状态 s，输出是每个动作的价值；输出节点数量等于动作空间大小 $|\mathcal{A}|$。
- 表格型方法每个状态单独存一行；网络则在状态之间共享参数，可泛化到未见过的状态。

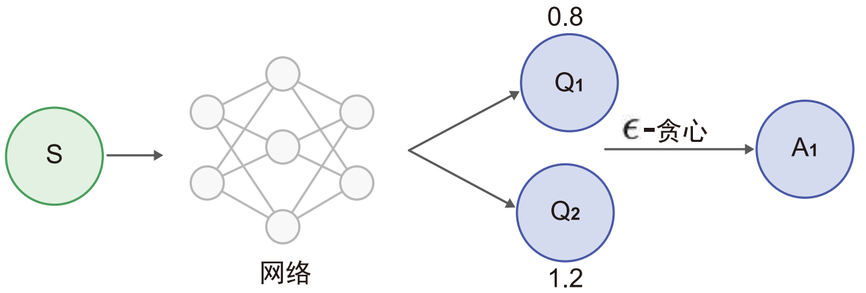


图 7.12　DQN 通过深度神经网络根据状态估计动作价值

DQN 的两项改进

用非线性函数估计动作价值并结合自举时，Q 学习会导致估计不稳定、有偏。深度 Q 网络（DQN）用两项改进使 Q 学习更稳定。

改进一

经验回放

数据收集时用 ϵ-贪心选动作；每个时间步的经验记为四元组 $e_t = (S_t, A_t, R_t, S_{t+1})$，存入经验回放缓冲区 $\mathcal{D} = \{e_1, e_2, \cdots, e_t\}$。

$\mathcal{D}$ 存储多个幕的经验，训练时从中随机采样。经验回放提高了数据效率，并平滑了数据分布。

改进二

目标网络周期性更新

估计的动作价值向目标动作价值优化。每隔参数 C 步，从深度 Q 网络复制参数 $\theta^- \leftarrow \theta$ 作为**目标 Q 网络**，并冻结其参数。

此改进避免了自举问题，提高训练稳定性，减少短期震荡。

两项改进分别针对“样本相关”与“目标漂移”两个不稳定来源：前者打散时间相关性，后者给自举目标一个固定的落点。

经验回放缓冲区

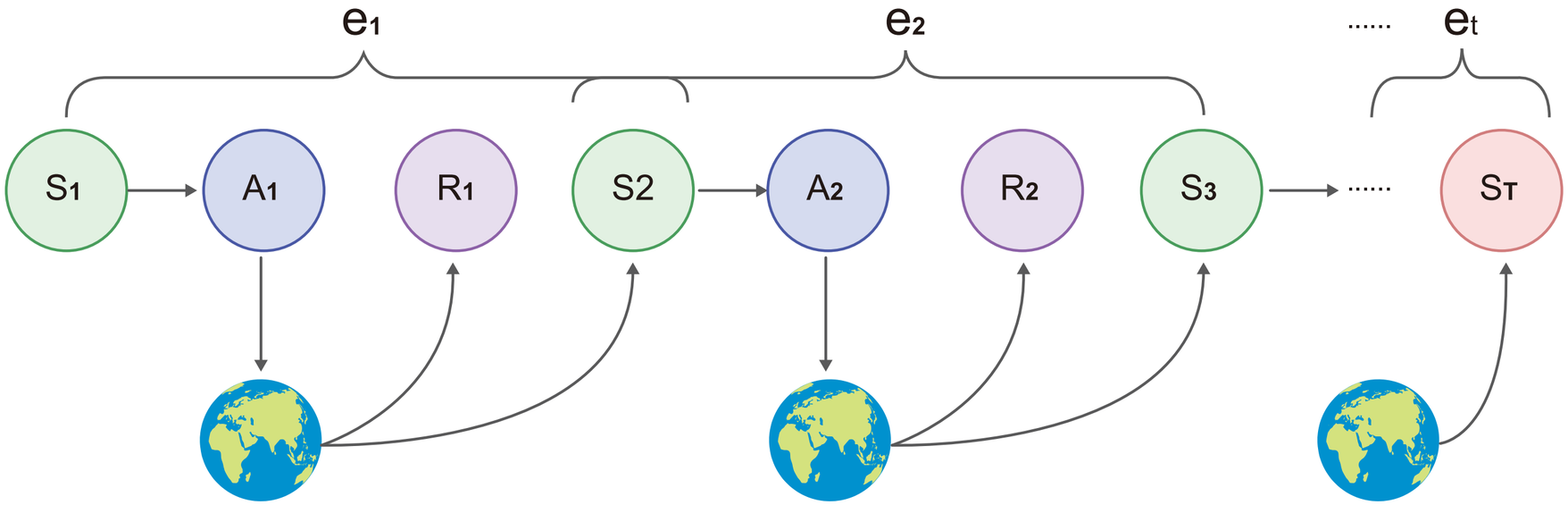


图 7.13　经验回放缓冲区存储时序差分数据

- 把一条轨迹上的 (S_t, A_t, R_t, S_{t+1}) 逐条压入缓冲区 $\mathcal{D}$，多条轨迹的经验混在一起存放。
- 训练时按均匀分布 $U(\mathcal{D})$ 随机采样小批量，而不是按时间顺序取用。
- 随机采样打破了相邻样本之间的强相关，使梯度方向更接近真实期望，也让同一批数据可以被反复使用。

DQN 的目标函数

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim U(\mathcal{D})}\left[\left(r + \gamma \max_{a'} q(s', a'; \theta^-) - q(s, a; \theta)\right)^2\right] \tag{7-51}$$

$U(\mathcal{D})$ 是经验回放缓冲区上的均匀分布；方括号内是"目标值 − 当前估计"的 TD 误差，外层取平方得到均方误差。

- 目标值中的 $\max_{a'} q(s', a'; \theta^-)$ 由**冻结的目标网络**给出，不参与当前步的梯度。
- $q(s, a; \theta)$ 由**在线 Q 网络**给出，梯度只对它回传。
- 用随机梯度下降更新 θ；参数更新达到 C 次时，执行 $\theta^- \leftarrow \theta$。

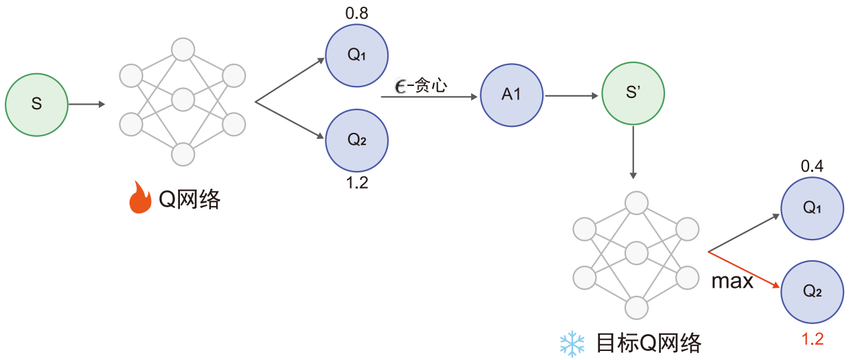


图 7.14　DQN 时序差分学习

算法 7　DQN

算法 7 DQN

输入： 初始化 Q 网络 θ，并以 θ 初始化目标 Q 网络，步长 $\alpha \in (0,1]$, $\epsilon \in [1,0]$，学习率 β，目标 Q 网络延迟更新步数 C

输出： 优化的动作价值估计网络 θ_*

for 每幕循环 **do**
  初始化状态 S
  以 ϵ-贪心根据 $Q(S_t, A_t; \theta)$ 选择动作 A_t
  执行动作 A_t，下一个状态 S_{t+1} 和奖励 R
  将 (S_t, A_t, R_t, S_{t+1}) 放入到经验回放缓存里 D
  if $|\mathcal{D}| >$ d **then**
    for 从经验 $\mathcal{D}$ 均匀采样的集合 (s, a, r, s') **do**
      在线获取动作价值 $Q(s, a; \theta)$
      获取目标 Q 网络的动作价值 $\arg\max_{a'} Q(s', a'; \theta^-)$
      TD(0) 误差为 $\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim U(\mathcal{D})} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]$
      $\theta \leftarrow \theta + \beta \Delta \mathcal{L}(\theta)$
    end for
  end if
  if 参数更新次数 $\geqslant C$ **then**
    $\theta^- \leftarrow \theta$
  end if
end for

DQN 与 Q 学习的实现差别

交互
ϵ-贪心采集
(S, A, R, S')

→

入池
写入 $\mathcal{D}$

→

随机采样
小批量 (s, a, r, s')

→

算损失
式 (7-51)

→

更新
θ，每 C 步同步 θ^{-}

- 在环境交互的同时优化策略，学习与采样并行推进。
- 通过经验回放重用历史数据，提高采样效率，并充分利用离线收集的数据，工程效率更高。
- 代价是内存占用与一条“数据延迟”：策略更新用的是稍早的策略采到的数据。

与在线 Q 学习相比，DQN 把“采样—更新”拆成两个异步环节：环境交互持续往池里写，训练循环反复从池里读。这样同一份交互数据可以被多次使用，样本效率随之提高。

与 Q 学习相同

目标都是 $r + \gamma \max_{a'} Q(s', a')$，都靠 ϵ-贪心采样。

与 Q 学习不同

用网络代替表格，用回放池代替即时样本，用目标网络代替即时目标。

多步时序差分

单步 TD 只关注相邻状态的价值更新关系，价值估计可能存在较大方差。计算未来多步的累积奖励可以降低方差，这就是多步时序差分方法。

具体做法是追踪一条动作链，把单步 TD 学习扩展为多步回报：根据接下来的 n 步计算 $G_t^{(n)}$（$n = 1, \ldots, \infty$）。

表 7.2　多步价值估计方法

n	n 步回报 $G_t^{(n)}$	方法类型
$n = 1$	$G_t^{(1)} = R_{t+1} + \gamma v(S_{t+1})$	单步 TD 学习
$n = 2$	$G_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 v(S_{t+2})$	2 步 TD 学习
$\vdots$	$\vdots$	$\vdots$
$n = n$	$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n v(S_{t+n})$	n 步 TD 学习
$\vdots$	$\vdots$	$\vdots$
$n = \infty$	$G_t^{(\infty)} = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$	蒙特卡洛（MC）估计

n 步时序差分的更新

$$v(S_t) \leftarrow v(S_t) + \alpha \left(G_t^{(n)} - v(S_t) \right) \quad (7\text{-}52)$$

广义的 n 步 TD 学习：把式 (7-46) 中的单步目标换成 n 步回报 $G_t^{(n)}$。$n = 1$ 退化为 TD(0)，$n = \infty$ 退化为 MC。

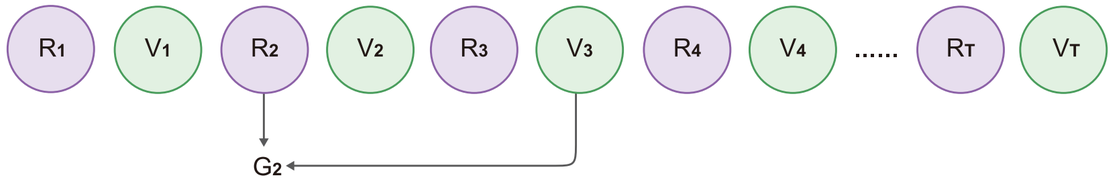


图 7.15　广义 n 步 TD 学习的价值函数更新

- $G_t^{(n)}$ 由 n 步真实奖励加上第 n 步处的价值估计组成，前段采样、后段自举。
- n 越大，自举越少、方差越大、偏差越小；n 越小则相反。
- $n = 1$ 时退化为式 (7-46) 的 TD(0)；n 取到幕末时退化为蒙特卡洛。

图中每个 R_i 都是实际观测到的奖励，末端的 V_T 则是估计值。所谓“多步”，就是让真实观测的那一段更长、让估计收尾的位置更靠后。

λ-回报

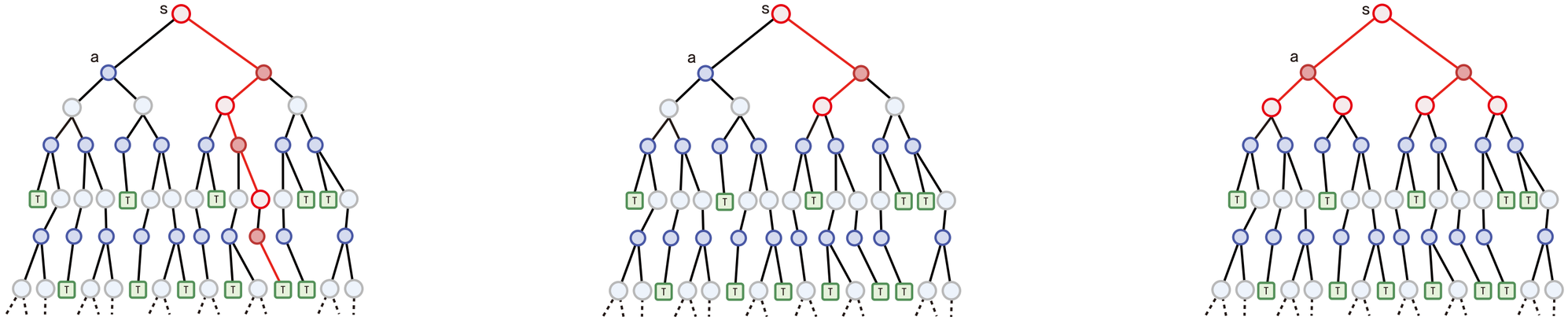
$G_t^{(n)}$ 提供了不同的精度-方差权衡，但如何确定最优的 n 是个关键问题。更好的方案是不固定 n，而是用指数衰减权重 λ^{n-1} 对不同步数的回报加权平均，$\lambda \in [0, 1]$。

$$G_t^{\lambda} = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)} \tag{7-53}$$

$(1-\lambda)$ 是归一化因子：因为 $\sum_{n=1}^{\infty} \lambda^{n-1} = \frac{1}{1-\lambda}$，乘上 $(1-\lambda)$ 后权重总和为 1。

- 未来奖励的不确定性更高，因此赋予较低权重；λ 控制衰减的快慢。
- 使用 λ-回报的 TD 学习方法称为 $\mathrm{TD}(\lambda)$：$\lambda = 0$ 退化为单步 TD，$\lambda = 1$ 退化为 MC。

三种价值估计方法的对比



(a) 多步时序差分学习　(b) 单步时序差分学习　(c) 期望时序差分学习

图 7.16　多步时序差分学习价值估计方法对比

- (a) 沿动作链展开多步，末端用价值估计收尾；(b) 只展开一步；(c) 在每一步对动作取期望而非采样。
- 三者的共同点：都在“采样多深”与“从哪里开始自举”之间做不同取舍。

三种方法的共性

基于价值函数的强化学习方法，核心在于利用价值函数指导策略改进。它们有以下共性。

共性一

评估是改进的基础

准确的价值函数评估是决策出更优策略的关键；策略改进的目标在很大程度上依赖评估的准确性。

共性二

回溯更新机制

通过回溯实现：求解贝尔曼方程、时序差分更新（如 SARSA）或多步时序差分更新，都是利用后续状态或状态-动作对的价值来更新当前估计。

共性三

无模型方法的优势

MC 与 TD 不依赖对环境状态转移概率的显式建模，而是通过采样经验或迭代更新估计价值，规避对复杂环境模型的直接求解。

依据 《大语言模型：训练、精调与对齐》第 7.5.7 节

方法的四个区分维度

维度一

同轨与离轨

取决于采样策略与优化策略是否相同。Q 学习与 DQN 是典型离轨方法：目标策略是贪心策略，而采样用的是 ϵ-贪婪策略。

维度二

探索与利用的平衡

要在探索未知区域（ϵ-贪婪中的随机探索）与利用当前最优动作之间取得平衡；经验回放（DQN）有助于更充分地利用已有数据。

维度三

估计偏差

首次访问的 MC 对期望回报无偏；TD 方法（含 Q 学习与 SARSA）因自举而产生有偏估计。

维度四

估计方差

期望 SARSA 比 Q 学习更新更稳定、方差更低；多步 TD 结合短期采样回报与长期估计，在偏差与方差之间取得折衷。

依据 《大语言模型：训练、精调与对齐》第 7.5.7 节

GPI 框架与方法边界

广义策略迭代（GPI）框架为许多基于价值函数的方法提供了理论基础，保证了特定条件下的收敛性。

已知模型

DP 评估 + 改进迭代收敛到最优

表格 + 无模型

Q 学习在满足条件时收敛到 q_*

函数近似

DQN 扩展到高维连续状态

- 传统价值方法围绕表格型表示求解；DQN 等方法通过函数逼近，自然地处理高维连续状态空间。
- 值函数方法既可用于模型已知的场景，也适用于环境未知的场景（MC、TD、Q 学习）。
- 核心挑战：巨大的状态-动作空间、环境模型未知、采样效率，以及函数逼近下的稳定性。

蒙特卡洛树搜索（MCTS）在 AlphaGo 中的应用是强化学习发展史上的重要里程碑：它结合深度学习（价值函数与策略网络）与 MCTS，在模拟中反复利用学到的价值与策略做前向搜索，显著提升具体状态下的决策质量。

思考题与实践题

1 说明马尔可夫性质的含义，并解释为什么它能把序贯决策问题的历史压缩为当前状态。

2 从回报的递归式 $G_t = R_{t+1} + \gamma G_{t+1}$ 出发，推导贝尔曼期望方程，并指出"自举"在哪一步出现。

3 比较蒙特卡洛与时序差分：二者分别需要多少数据才能更新一次价值估计，偏差与方差各有什么特点。

4 写出 SARSA 与 Q 学习的更新式，说明同轨策略与离轨策略的差别，并解释为什么 Q 学习能收敛到 q_*。

5 DQN 的经验回放与目标网络分别解决了什么不稳定问题？若去掉其中之一，训练会出现什么现象？

6 推导 REINFORCE 的样本梯度，说明 $\nabla_{\boldsymbol{\theta}} \ln \pi_{\boldsymbol{\theta}}$ 的作用，以及基线为何不改变梯度的期望。

7 比较 TRPO 的 KL 约束与 PPO 的裁剪目标：二者都在限制策略更新幅度，实现方式与代价有何不同。

8 把语言模型的生成过程写成 MDP，指出状态、动作、转移与奖励，并说明 RLHF 中奖励为何只在 <EOS> 处非零。

依据 《大语言模型：训练、精调与对齐》第 7 章

本章算法的代码实现

每个算法一个文件、一个类，3×3 网格世界讲表格型方法，CartPole-v1 讲需要神经网络的方法。

文件	算法	教材位置	环境
dp.py	策略评估、策略改进、策略迭代、值迭代	7.3 节	网格（已知模型）
mc.py	首次访问 MC 估计、试探性出发 MC 控制	7.4 节	网格
td.py	表格型 TD(0) 策略评估	7.5.1 节	网格
sarsa.py	SARSA、期望 SARSA	7.5.2 节	网格
q_learning.py	Q 学习	7.5.3 节	网格
td_lambda.py	n 步回报、λ-回报、TD(λ)	7.5.4 节	网格
dqn.py	DQN：经验回放 + 目标网络	7.5.5 节	CartPole-v1
pg.py	REINFORCE、带基线、离轨策略	7.6.3 节	CartPole-v1
actor_critic.py	演员-评论家	7.7.1 节	CartPole-v1
gae.py	广义优势估计（纯函数，讲计算过程）	7.8.2 节	—
ppo.py	PPO-Clip，内部实现 GAE	7.7.3 节	CartPole-v1

配套 envs/grid_world.py 提供 deterministic / slip / wind 三种转移场景，bellman.py、transition.py、why_v_is_harder.py 给出本章各式的数值实例。每个文件都可 python 文件名.py 独立运行。